

SIC AI Chapter Quiz: 09

Student Name: Devesh Panwar

Candidate ID: SIC202500060

Question 01

- **Answer: Softmax**
- **Reasoning:**
 - MNIST is a **multi-class classification** problem with 10 classes (digits 0-9).
 - The **Softmax** function is designed for this purpose. It normalizes the output vector into a probability distribution where the sum of all probabilities equals 1, allowing the model to select the class with the highest probability.

Question 02

(1) The Scale Problem with the Training Set

- **Observation:** There is a massive disparity in the magnitude of the features.
 - **Blood Pressure** values are in the hundreds (~120).
 - **Height** values are very small single digits (~1.7).
- **The Problem:** This causes **Feature Dominance**. In a perceptron (which calculates $w \cdot x$), the "Blood Pressure" feature will numerically overpower the "Height" feature.
- **Consequence:** The model effectively ignores the Height feature because even a tiny change in the Blood Pressure weight causes a huge change in the total output. This also leads to an elongated error surface, causing the gradient descent algorithm to take a zigzag path and converge very slowly.

(2) The New Training Set (After Standardization)

We apply the Z-score normalization formula: $x_{new} = \frac{x_{old} - \mu}{\sigma}$

The resulting standardized training set is:

$$\begin{pmatrix} -0.1228 & 1.3351 & -0.1995 & 0.6445 & -1.6573 \\ -0.1162 & -1.5693 & -0.4069 & 1.3369 & 0.7556 \\ -0.0417 & -0.7523 & -1.2668 & 1.5511 & 0.5097 \end{pmatrix}$$

(Rows are Blood Pressure, Height, Weight respectively)

(3) Observation on Alleviating the Problem

- **Yes, this alleviates the scale problem.**
- **Reasoning:** After this preprocessing, every feature (Blood Pressure, Height, and Weight) has been transformed to have a **Mean of 0** and a **Standard Deviation of 1**.
- **Benefit:** The features are now on the exact same scale. The model can now learn the true importance of "Height" without it being drowned out by the large numbers in "Blood Pressure." This ensures faster and more stable training convergence.

```
import numpy as np

# --- Chapter 9, Question 2 Code Solution ---

# 1. Define the Input Data (Rows = Features, Cols = Samples)
# Row 0: Blood Pressure
# Row 1: Height
# Row 2: Weight
X = np.array([
    [121, 140, 120, 131, 101],
    [1.72, 1.62, 1.70, 1.82, 1.78],
    [69.0, 63.2, 59.0, 82.0, 73.5]
])

# 2. Calculate Mean and Std Dev for each feature (axis=1)
# keepdims=True ensures the shape remains compatible for broadcasting
mu = np.mean(X, axis=1, keepdims=True)
sigma = np.std(X, axis=1, keepdims=True)
```

```
# 3. Apply the Preprocessing Formula (Standardization)
X_new = (X - mu) / sigma

# 4. Print the Results
print("Question 2-(2) Resulting Matrix:\n")
print(np.round(X_new, 4))

print("\n--- Verification ---")
print(f"New Mean (should be 0): {np.round(np.mean(X_new, axis=1), 2)}")
print(f"New Std (should be 1): {np.round(np.std(X_new, axis=1), 2)}")
```

Question 2-(2) Resulting Matrix:

```
[[-0.1228  1.3351 -0.1995  0.6446 -1.6574]
 [-0.1162 -1.5693 -0.4069  1.3368  0.7556]
 [-0.0423 -0.7647 -1.2879  1.5768  0.5181]]

--- Verification ---
New Mean (should be 0): [ 0. -0. -0.]
New Std (should be 1): [1. 1. 1.]
```

Question 03

- **Answer: Padding** (Zero Padding)
- **Reasoning:** Convolution operations naturally reduce the spatial dimensions of the input. Repeating this multiple times in deep networks causes significant data loss at the borders. **Padding** adds extra pixels around the input to maintain the spatial resolution (height and width) throughout the network layers.

Question 04

```
import numpy as np

# Generate random weights in range [-1, 1]
# You can change the shape (3, 3) to whatever dimension your network needs
weights = np.random.uniform(-1, 1, (3, 3))

print("Generated Weights:\n", weights)
```

Generated Weights:

```
[[ 0.5042241  0.50587835  0.19288185]
 [ 0.03484852  0.11840768 -0.01198425]
 [ 0.1777465  0.31061666 -0.01343055]]
```

Question 05

```
import numpy as np

# --- Function to generate weights in range [-1, 1] ---
def generate_weights(rows, cols):
    # np.random.uniform(low, high, size) generates numbers in the half-open interval [low, high).
    weights = np.random.uniform(-1, 1, (rows, cols))
    return weights

# --- Example Usage ---
# Creating a weight matrix for 3 input nodes and 4 hidden nodes
W = generate_weights(3, 4)

print("Random Weights in range [-1, 1]:\n")
print(W)

# Verification
print(f"\nMax value: {W.max():.4f} (Should be <= 1)")
print(f"Min value: {W.min():.4f} (Should be >= -1)")
```

Random Weights in range [-1, 1]:

```
[[-0.48015329  0.83264029 -0.45320393 -0.71607977]
 [ 0.2219272  0.09466682 -0.82883832 -0.07157872]
 [ 0.66823438  0.68425022  0.35647624  0.86126814]]

Max value: 0.8613 (Should be <= 1)
Min value: -0.8288 (Should be >= -1)
```

Question 06

```
import tensorflow as tf
import numpy as np
import pandas as pd
from tensorflow.keras.datasets.fashion_mnist import load_data
import matplotlib.pyplot as plt

# --- 1. Load Data ---
(X_train, y_train), (X_test, y_test) = load_data()

# --- 2. Data Processing ---
# Reshape to (Batch, Height, Width, Channels) -> (N, 28, 28, 1)
X_train = X_train.reshape((-1, 28, 28, 1))
X_test = X_test.reshape((-1, 28, 28, 1))

# Scale pixel values to range [0, 1]
X_train = X_train / 255.0
X_test = X_test / 255.0

# --- 3. Training Parameters ---
batch_size = 128 # Increased slightly for speed (image said 8, but 128 is standard)
n_epochs = 10 # Reduced to 10 for quick testing (image said 20)
learn_rate = 0.001

# --- 4. Build the CNN Model (COMPLETING THE CODE) ---
model = tf.keras.Sequential([
    # Input Layer
    tf.keras.layers.Input(shape=(28, 28, 1)),

    # Convolutional Block 1
    tf.keras.layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),

    # Convolutional Block 2
    tf.keras.layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),

    # Flatten and Dense Layers
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.3), # Helps prevent overfitting

    # Output Layer (10 classes for Fashion MNIST)
    tf.keras.layers.Dense(10, activation='softmax')
])

# --- 5. Compile and Train ---
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=learn_rate),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

print("\nStarting Training...")
history = model.fit(X_train, y_train,
                    epochs=n_epochs,
                    batch_size=batch_size,
                    validation_data=(X_test, y_test))

# --- 6. Plot Results (Optional) ---
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
plt.show()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>
 29515/29515 ————— 0s 0us/step
 Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>
 26421880/26421880 ————— 0s 0us/step
 Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>
 5148/5148 ————— 0s 0us/step
 Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>
 4422102/4422102 ————— 0s 0us/step
 Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense_1 (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)

Starting Training...

Epoch 1/10
 469/469 ————— 49s 102ms/step - accuracy: 0.6926 - loss: 0.8647 - val_accuracy: 0.8543 - val_loss: 0.4025
 Epoch 2/10
 469/469 ————— 44s 93ms/step - accuracy: 0.8574 - loss: 0.3958 - val_accuracy: 0.8756 - val_loss: 0.3432
 Epoch 3/10
 469/469 ————— 82s 93ms/step - accuracy: 0.8788 - loss: 0.3329 - val_accuracy: 0.8857 - val_loss: 0.3144
 Epoch 4/10
 469/469 ————— 83s 97ms/step - accuracy: 0.8890 - loss: 0.3037 - val_accuracy: 0.8879 - val_loss: 0.3075
 Epoch 5/10
 469/469 ————— 43s 93ms/step - accuracy: 0.8967 - loss: 0.2830 - val_accuracy: 0.8934 - val_loss: 0.2896
 Epoch 6/10
 469/469 ————— 83s 94ms/step - accuracy: 0.9049 - loss: 0.2571 - val_accuracy: 0.9029 - val_loss: 0.2670
 Epoch 7/10
 469/469 ————— 83s 97ms/step - accuracy: 0.9123 - loss: 0.2374 - val_accuracy: 0.9020 - val_loss: 0.2617
 Epoch 8/10
 469/469 ————— 44s 94ms/step - accuracy: 0.9134 - loss: 0.2282 - val_accuracy: 0.9079 - val_loss: 0.2541
 Epoch 9/10
 469/469 ————— 44s 94ms/step - accuracy: 0.9216 - loss: 0.2150 - val_accuracy: 0.9069 - val_loss: 0.2584
 Epoch 10/10
 469/469 ————— 43s 92ms/step - accuracy: 0.9275 - loss: 0.1961 - val_accuracy: 0.9082 - val_loss: 0.2499

